UNIVERSIDADE REGIONAL DE BLUMENAU
CENTRO DE CIÊNCIAS EXATAS E NATURAIS
DEPARTAMENTO DE SISTEMAS E COMPUTAÇÃO
PROFESSOR ANDRÉ FELIPE BÜRGER
PROGRAMAÇÃO ORIENTADA A OBJETOS

Lista de Revisão: pós-Avaliação 02

Esta lista revisa os pontos em que a turma mais perdeu nota na Avaliação 02. Leia as orientações abaixo antes de começar.

- Faça as partes na ordem: a Parte A fixa cada conceito isolado, a Parte B mostra os erros que apareceram na prova e a Parte C junta tudo, no mesmo formato da prova.
- **Compile a cada classe que terminar.** Na prova, muitos projetos não compilaram por código fora de método, chave sobrando ou método declarado dentro de outro.
- Entregue um único projeto Java, com um pacote para cada parte: `aquecimento`, `cacaaoserros` e `setup`.
- As respostas escritas vão como comentário no próprio código, perto do trecho a que se referem.

Conceitos revisados

Conceito	Onde aparece
Membros static: o que é da classe e o que é do objeto	A1, B5, C
Variável do tipo da abstração e `instanceof` com interface	A2, B8, C
Composição, agregação e herança	A3, C
Regras da agregação: limite, ordem das validações, tipo da exceção, remoção	B1 a B4, C
Cálculo que reaproveita outro método da própria classe	B6, C
`@Override` e `toString()`	B7, C
Detalhes do main: for-each, `getNome()`, try/catch	B8, C

Parte A. Aquecimento

A1. Senhas de atendimento (static)

Pense no totem de senhas do banco: cada senha sai com um número novo, e o totem sabe quantas senhas já emitiu. O número é de cada senha; a contagem é do totem inteiro.

Crie a classe `Senha` com:

a) Os atributos `numero` (int) e `setor` (String), privados, e o contador privado `totalEmitidas` (int).

b) O construtor `Senha(String setor)`. Ele **não** recebe o número: o número é gerado automaticamente, 1, 2, 3, e assim por diante.

c) Se `setor` for null ou vazio, o construtor lança `IllegalArgumentException` com a mensagem "Setor obrigatório". Uma senha inválida **não pode consumir número**: se a 2ª tentativa falhar, a próxima senha válida ainda é a de número 2.

d) Os getters `getNumero()`, `getSetor()` e `getTotalEmitidas()`.

e) O `toString()` no formato `2 - Caixa`.

No `main`, chame `Senha.getTotalEmitidas()` **antes** de criar qualquer senha (deve exibir 0). Depois crie três senhas válidas, tente criar uma inválida dentro de try/catch, crie mais uma válida e exiba todas. Saída esperada:

```
Emitidas: 0
1 - Caixa
2 - Gerência
3 - Caixa
Erro: Setor obrigatório
4 - Caixa
Emitidas: 4
```

Responda em comentário no código:

- O que precisa ser `static` para o código acima funcionar, e por quê? Remova o `static` do contador, rode de novo e explique o que mudou na saída.
- Por que `getTotalEmitidas()` também precisa ser `static`? O que acontece com a chamada `Senha.getTotalEmitidas()` se ele não for?
- Em que linha do construtor o contador é incrementado, e por que essa posição garante o item c)?

A2. Catálogo de streaming (abstração e interface)

Um catálogo tem a classe abstrata `Midia`, com `descricao`, `precoBase` e o método abstrato `double calcularPreco()`. `Filme` cobra o `precoBase`. `Serie` cobra `precoBase` x `temporadas` e implementa a interface `Legendavel`, que tem o método `double calcularCustoLegenda()` (10% do preço calculado). Implemente as quatro classes.

Um colega escreveu este main:

```
Filme f = new Filme("Interestelar", 20.0);
Serie s = new Serie("Dark", 15.0, 3);
ArrayList<Midia> catalogo = new ArrayList<>();
catalogo.add(f);
catalogo.add(s);

double totalLegenda = 0;
for (Midia m : catalogo) {
    if (m instanceof Serie) {
        totalLegenda += ((Serie) m).calcularCustoLegenda();
    }
}
```

a) Reescreva o main declarando as variáveis com o tipo `Midia`. Explique em comentário o que você ganha com isso.

b) Crie a classe `Documentario`, que estende `Midia`, cobra `precoBase` e **também** implementa `Legendavel`. Adicione um documentário ao catálogo e rode o código do colega. O custo de legenda do documentário entrou no total?

c) Corrija o laço para que qualquer mídia legendável entre no total, inclusive classes que ainda nem existem. Que tipo vai no `instanceof`, e para que tipo é o cast?

A3. É um, tem um ou guarda um? (composição, agregação e herança)

Para cada par, diga se a relação é herança, composição ou agregação e justifique em uma frase:

Par	Relação	Por quê
Gerente e Funcionario		
Carro e Motor (o motor é criado com o carro e não existe sem ele)		
Playlist e Musica		
Pedido e Endereco de entrega		

Agora corrija as duas versões de `Pedido` abaixo, similares a prova:

```
// Versão 1
public class Pedido extends Endereco {
    public Pedido(Cliente comprador, String rua, int numero) {
        super(rua, numero);
    }
}

// Versão 2
public class Pedido {
    private String rua;
    private int numero;
    public Pedido(Cliente comprador, String rua, int numero) {
        Endereco entrega = new Endereco(rua, numero);
        this.rua = rua;
        this.numero = numero;
    }
}
```

Na sua correção, o `Pedido` cria o `Endereco` no construtor, guarda-o num atributo privado chamado `entrega` e expõe só `getRuaEntrega()` e `getNumeroEntrega()`. Não pode haver `getEntrega()` nem `setEntrega()`. Explique em comentário por que a Versão 1 é um erro de modelagem mesmo compilando.

Parte B. Caça aos erros

Todos os trechos abaixo compilam e todos estão errados. Cada um reproduz um erro que apareceu na prova, já adaptado às classes da Parte C (o orçamento aceita no máximo **5 itens**). Para cada trecho, responda em comentário: (a) qual é o erro, (b) um caso de teste concreto que o mostra, com o resultado obtido e o esperado, e (c) a versão corrigida.

B1. Limite

```
if (itens.size() > 5) {
    throw new IllegalStateException("Orçamento lotado");
}
itens.add(item);
```

B2. Tipo da exceção

```
if (itens.contains(item)) {
    throw new IllegalArgumentException("Item já incluído no orçamento");
}
```

Dica: compare o que significa `IllegalArgumentException` (o argumento é inválido) com `IllegalStateException` (o argumento é válido, mas o estado do objeto impede a operação). Em qual dos dois casos está um item repetido? E um item null?

B3. Ordem das validações

```
if (item == null) throw new IllegalArgumentException("Item inválido");
if (itens.size() == 5) throw new IllegalStateException("Orçamento lotado");
if (itens.contains(item)) throw new IllegalStateException("Item já incluído no orçamento");
```

O enunciado pede null, depois repetido, depois lotado. Monte o cenário em que esta versão devolve a mensagem errada.

B4. Remoção

```
public void removerItem(ItemSetup item) {
    for (ItemSetup i : itens) {
        if (i != item) {
            throw new IllegalStateException("Item não encontrado no orçamento");
        }
    }
    itens.remove(item);
}
```

Teste com três itens na lista, removendo o segundo. Teste também com a lista vazia. Dica para a correção: o método `remove` de `ArrayList` devolve `boolean`.

B5. Contador

```
public ItemSetup(String descricao, double precoBase) {
    totalItens++;
    codigo = totalItens;
    if (precoBase <= 0) {
        throw new IllegalArgumentException("Preço base inválido");
    }
    this.descricao = descricao;
    this.precoBase = precoBase;
}
```

Crie um item válido, tente um inválido e crie outro válido. Que códigos saem? Que códigos deveriam sair?

B6. Base do cálculo

```
@Override
public double calcularGarantiaEstendida() {
    return getPrecoBase() * quantidade * 0.15;
}
```

O enunciado pede 15% do **preço calculado**. Para 3 monitores de R$ 1.200,00, quanto este método devolve, e quanto deveria devolver? Reescreva usando um método que a classe já tem.

B7. toString sem @Override

```
public String tostring() {
    return getCodigo() + " - " + getDescricao() + " - R$ " + calcularPreco();
}
```

Rode `System.out.println(item)`. O que aparece? Coloque `@Override` em cima do método e compile. O que o compilador diz? Explique por que `@Override` vale também para métodos herdados de `Object`, e não só para métodos abstratos ou de interface.

B8. main

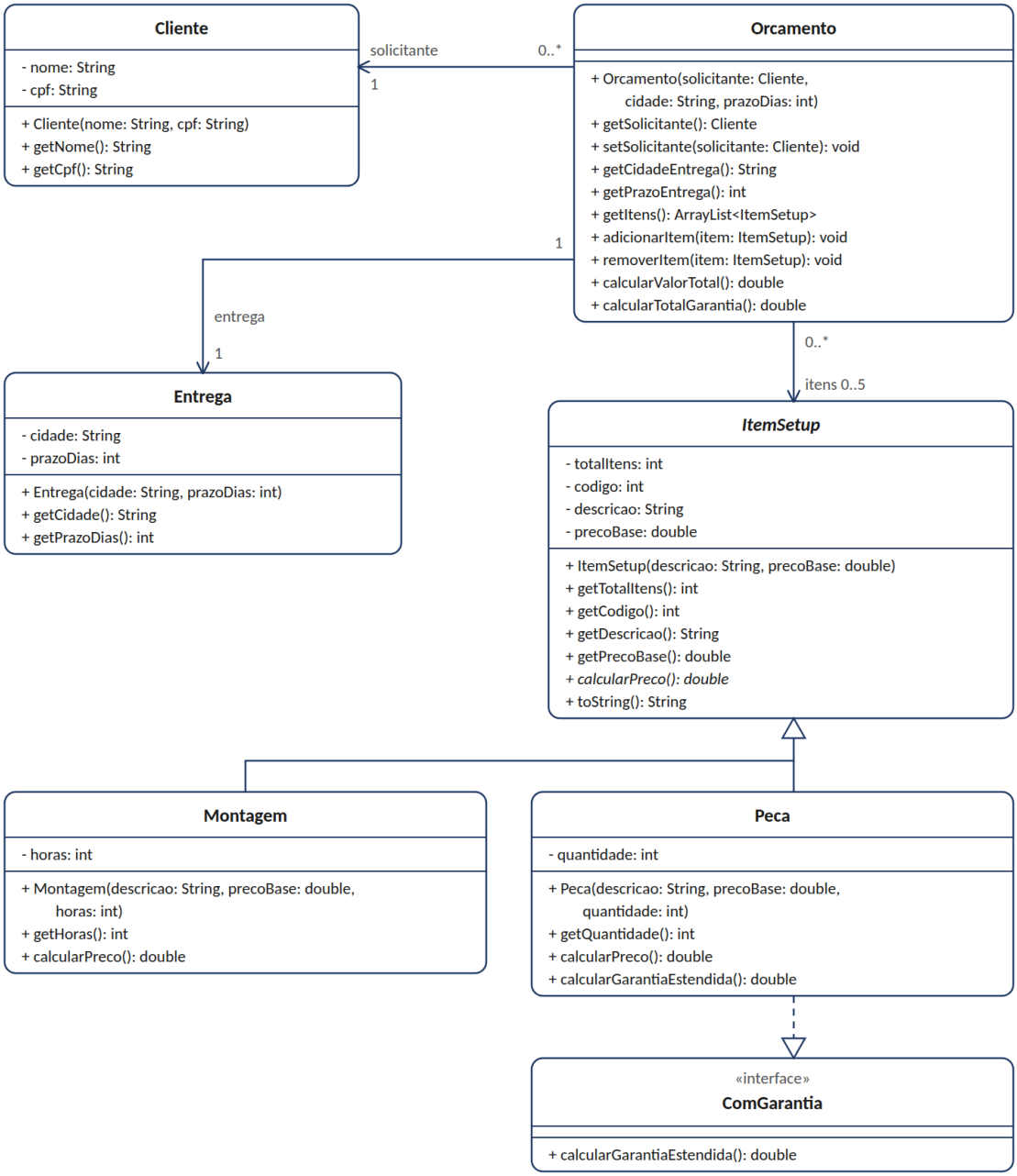
```
Peca placa = new Peca("Placa de vídeo", 2500.0, 1);
orcamento.adicionarItem(placa);
System.out.println("Solicitante: " + orcamento.getSolicitante());
System.out.println(orcamento.getItens());
orcamento.adicionarItem(placa);
```

Aqui há quatro problemas. Encontre todos: o tipo da variável, o que sai na linha do solicitante, a forma de exibir os itens e o que acontece na última linha.

Parte C. Orçamento de setup gamer

Uma loja de informática monta setups gamer sob encomenda. Cada orçamento pertence a um cliente, tem uma entrega com cidade e prazo e reúne até 5 itens: peças, que têm garantia estendida, e serviços de montagem, que não têm. Implemente as classes do diagrama abaixo no pacote `setup`, na mesma estrutura da prova.

Diagrama de classes (pacote setup)



O diagrama não marca o que é static de propósito: decidir isso faz parte do exercício.

UNIVERSIDADE REGIONAL DE BLUMENAU
CENTRO DE CIÊNCIAS EXATAS E NATURAIS
DEPARTAMENTO DE SISTEMAS E COMPUTAÇÃO
PROFESSOR ANDRÉ FELIPE BÜRGER
PROGRAMAÇÃO ORIENTADA A OBJETOS

C1. Cliente, parte composta e associação

a) `Cliente` com `nome` e `cpf` privados, construtor e getters.

b) `Entrega` com `cidade` (String) e `prazoDias` (int) privados, construtor e getters.

c) Em `Orcamento`, a associação com `Cliente` fica num atributo privado chamado `solicitante`. O `setSolicitante()` lança `IllegalArgumentException` com "Solicitante obrigatório" quando recebe null, e o construtor passa pela mesma validação.

d) A `Entrega` é **composição**: o construtor `Orcamento(Cliente solicitante, String cidade, int prazoDias)` cria o objeto `Entrega` e o guarda no atributo privado `entrega`. Não pode existir getter nem setter que devolva ou receba o objeto `Entrega`. Quem usa o orçamento só enxerga `getCidadeEntrega()` e `getPrazoEntrega()`, que devolvem os dados da parte.

C2. Hierarquia

a) A interface `ComGarantia` com o método `double calcularGarantiaEstendida()`.

b) A classe abstrata `ItemSetup`, com `codigo`, `descricao` e `precoBase` privados, o método abstrato `double calcularPreco()` e o contador `totalItens`. O código é gerado automaticamente (1, 2, 3...) e **não** é recebido por parâmetro. Decida o que é static aqui, tanto entre os atributos quanto entre os métodos.

c) O construtor de `ItemSetup` lança `IllegalArgumentException` com "Preço base inválido" quando `precoBase` for menor ou igual a zero, **antes** de mexer no contador.

d) O `toString()` de `ItemSetup` devolve `codigo - descricao - R$ preco`, com o preço obtido por `calcularPreco()`. Exemplo: `1 - Placa de vídeo - R$ 2500.0`.

e) `Montagem` estende `ItemSetup` e reaproveita o construtor da superclasse com `super(...)`. Se `horas` for menor ou igual a zero, lança `IllegalArgumentException` com "Quantidade de horas inválida". O preço é `precoBase` x `horas`.

f) `Peca` estende `ItemSetup`, implementa `ComGarantia` e reaproveita o construtor com `super(...)`. Se `quantidade` for menor ou igual a zero, lança `IllegalArgumentException` com "Quantidade inválida". O preço é `precoBase` x `quantidade`, com 10% de desconto a partir de 3 unidades. A garantia estendida é 15% do **preço calculado**, com o desconto já aplicado.

g) `@Override` em todos os métodos que sobrescrevem ou implementam outro, `toString()` incluído.

C3. Agregação e regras do orçamento

a) O atributo `itens` é um `ArrayList<ItemSetup>` instanciado. `getItens()` devolve a lista, e não existe setter da lista.

b) `adicionarItem(ItemSetup item)` valida nesta ordem:

1. item null: `IllegalArgumentException` com "Item inválido";
2. item já presente: `IllegalStateException` com "Item já incluído no orçamento";
3. lista com 5 itens: `IllegalStateException` com "Orçamento lotado".

c) `removerItem(ItemSetup item)` tira o item da lista. Se o item não estiver nela, lança `IllegalStateException` com "Item não encontrado no orçamento", inclusive com a lista vazia.

d) `calcularValorTotal()` soma `calcularPreco()` de todos os itens por chamada polimórfica, sem `instanceof`.

e) `calcularTotalGarantia()` soma a garantia estendida só dos itens que têm garantia. Use `instanceof` com a **interface** e faça o cast para a interface. Por que não para `Peca`? Responda em comentário.

C4. Programa principal

Na classe `Principal`, com o método `main`:

a) Exiba `ItemSetup.getTotalItens()` antes de criar qualquer item.
b) Crie a cliente Marina Souza (CPF 123.456.789-00) e um orçamento com entrega em Blumenau, em 7 dias.
c) Crie os três itens em variáveis do tipo **`ItemSetup`**, nunca `Peca` ou `Montagem`:
   1. Placa de vídeo: peça de R$ 2500.0, 1 unidade;
   2. Monitor 27 polegadas: peça de R$ 1200.0, 3 unidades;
   3. Montagem e cabeamento: montagem de R$ 80.0 por hora, 2 horas.
d) Adicione os três ao orçamento.
e) Exiba o **nome** da solicitante e a cidade e o prazo de entrega.
f) Percorra `getItens()` com for-each, exibindo cada item.
g) Exiba o valor total e o total da garantia estendida.
h) Dentro de try/catch, tente adicionar a placa de vídeo de novo e exiba a mensagem da exceção. Faça o mesmo com uma montagem de 0 horas.
i) Exiba `ItemSetup.getTotalItens()` de novo.

Saída esperada:

```
Itens criados: 0
Solicitante: Marina Souza
Entrega: Blumenau, em 7 dias
1 - Placa de vídeo - R$ 2500.0
2 - Monitor 27 polegadas - R$ 3240.0
3 - Montagem e cabeamento - R$ 160.0
Valor total: R$ 5900.0
Garantia estendida: R$ 861.0
Erro: Item já incluído no orçamento
Erro: Quantidade de horas inválida
Itens criados: 4
```

A última linha mostra 4, e não 3. Explique em comentário por que a montagem inválida consumiu um código, mesmo com o `ItemSetup` validando antes de incrementar. Compare com o item c) da A1: o que é diferente entre os dois casos?

Bônus: pra quem quer ir além

A loja passou a vender licenças de software, que também têm garantia estendida. Crie a classe `Software`, que estende `ItemSetup`, implementa `ComGarantia` e tem o atributo `licencas`. O preço é `precoBase` x `licencas`, e a garantia é 15% do preço calculado.

A regra do desafio: **você não pode alterar nenhuma linha de** `Orcamento`. No main, adicione um Sistema operacional de R$ 300.0 com 2 licenças, depois dos outros três itens. O valor total deve passar para R$ 6500.0 e a garantia estendida para R$ 951.0.

Rode o main com a sua versão de `calcularTotalGarantia()`. Se a garantia não chegou a R$ 951.0, o que isso diz sobre o `instanceof` que você usou? Responda em comentário.

Checklist antes de entregar

Confira cada item no seu código. Todos eles custaram pontos na prova.

- ☐ O projeto compila inteiro, e eu compilei a cada classe terminada.
- ☐ Nenhum código está solto no corpo da classe: todo `if`, laço e cálculo está dentro de um construtor ou método.
- ☐ O contador é static, o `getTotalItens()` é static e o código não é recebido por parâmetro.
- ☐ A validação do preço base vem antes do incremento do contador.
- ☐ O todo cria a parte no construtor, guarda num atributo privado com o nome do papel e não tem getter nem setter dela.
- ☐ O limite usa o tamanho da lista e recusa o 6º item.
- ☐ As validações estão na ordem null, repetido, lotado, com `IllegalArgumentException` só para o null.
- ☐ A remoção de um item ausente lança exceção, inclusive com a lista vazia.
- ☐ A garantia usa `calcularPreco()`, e não o `precoBase`.
- ☐ O `instanceof` e o cast usam a interface, e não a classe concreta.
- ☐ Todo método que sobrescreve ou implementa outro tem `@Override`, inclusive o `toString()`.
- ☐ No main, as variáveis são do tipo `ItemSetup`, os itens saem num for-each e o nome da solicitante aparece com `getNome()`.
- ☐ A inclusão repetida está dentro de try/catch e o catch exibe a mensagem.
- ☐ As mensagens das exceções estão iguais às do enunciado.

Bons estudos!